MySQL 索引的最左前缀匹配原则是什么？ - 面试鸭 | 2026最新面试题+详细答案解析

> ## Excerpt
>
> 2026最新后端、MySQL、数据库面试题，包含详细答案解析和练习题，43928+程序员已学习，已帮助10万+程序员通过校招/社招/实习面试，拿到offer！

回答重点

MySQL 索引的**最左前缀匹配原则**是指使用联合索引时，查询条件必须从索引的最左列开始匹配。联合索引包含多个列，查询时必须先有第一个列的条件，然后是第二个列，以此类推。

这是因为联合索引在 B+ 树中的排列方式是"从左到右"的顺序。比如联合索引 `(first_name, last_name, age)` 会先按 first_name 排序，first_name 相同再按 last_name 排序，last_name 相同再按 age 排序。

MySQL 查找时会优先用 first_name 作为匹配依据，然后依次用 last_name 和 age。跳过最左侧字段，后面的列在 B+ 树中是无序的，压根没法利用索引快速定位。

扩展知识

匹配与不匹配的场景

假设当前表有一个联合索引 `(a, b, c)`。

符合最左匹配原则的查询条件：

```
▼sql复制代码where a = 1;
where a = 1 and b = 2;
where a = 1 and b = 2 and c = 3;
```

不符合最左匹配原则的查询条件：

```
▼sql复制代码where b = 2;
where c = 3;
where b = 2 and c = 3;
```

再看这个查询：

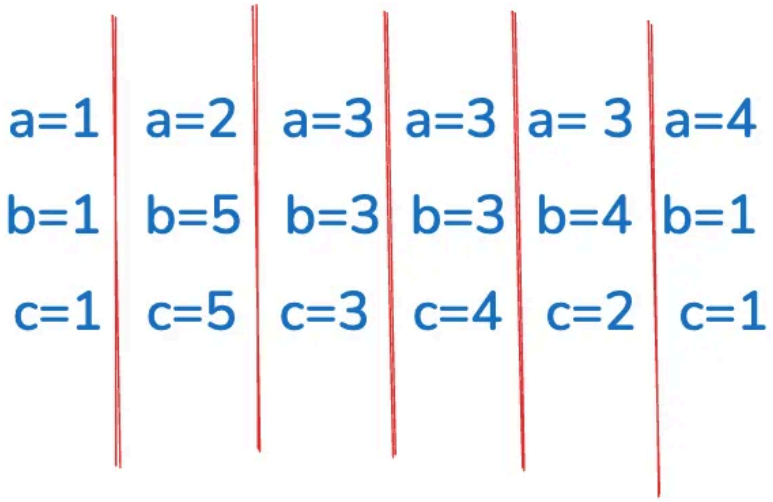
```
▼sql复制代码where a = 1 and c = 3;
```

这个查询可以用上联合索引 `(a, b, c)`，但在 MySQL 5.6 之前只能利用 `a = 1` 这个条件过滤。MySQL 5.6 版本引入了**索引下推**优化，可以在引擎层查到 `a = 1` 的数据后，直接用 c 过滤掉不符合的数据，减少回表次数。

- [596. MySQL 的索引下推是什么？]

范围查询的特殊规则

理解范围查询之前，得先搞清楚联合索引的排序规则。联合索引 `(a, b, c)` 在 B+ 树中的排序是：先按 a 的值排序，a 相同再按 b 排序，b 相同再按 c 排序。



面试鸭
mianshiya.com

遇到 `>`、`<` 这种范围查询，会停止匹配。

```
▼sql复制代码where a > 1 and b = 2 and c = 3;
```

所谓的停止匹配指的是：只有 a 能用上联合索引，b 和 c 用不上。因为 a 经过范围查询筛选后，不同 a 值之间的 b 和 c 是无序的。比如 a = 2 的数据和 a = 3 的数据，它们的 b 值没有任何排序关系，自然没法走索引。

遇到 `>=`、`<=`、`BETWEEN`、前缀 like `xx%` 这种范围查询，不会停止匹配。因为这些查询包含等值判断，可以直接定位到某条数据，然后往后扫描。

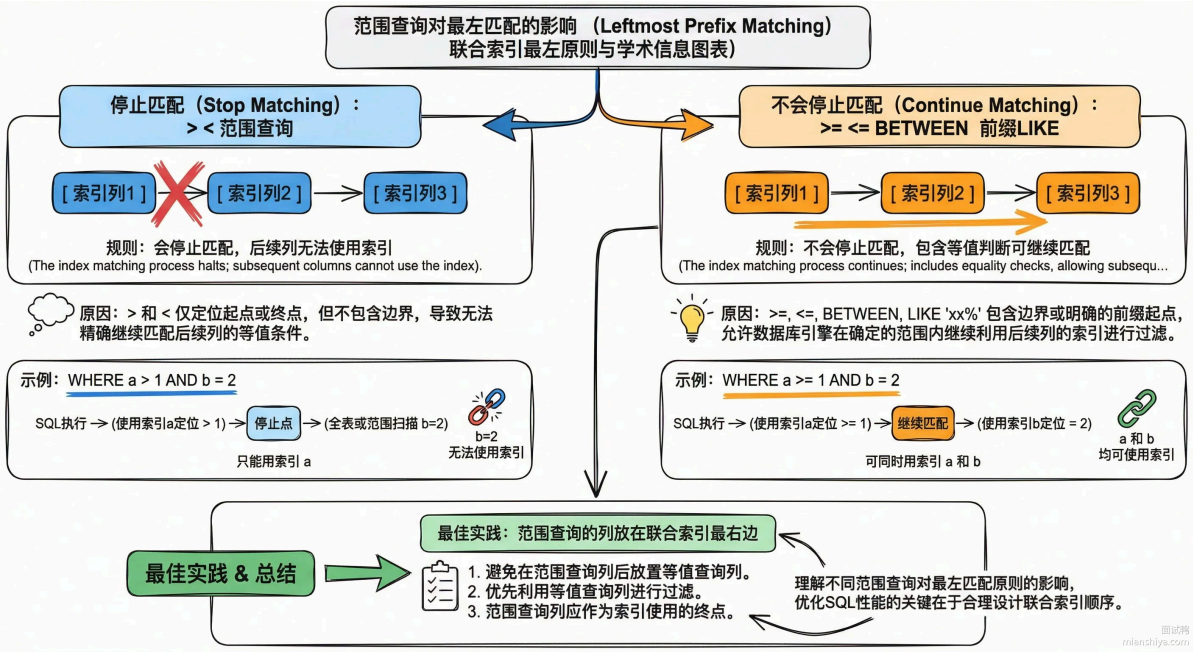
```
▼sql复制代码where a >= 1 and b = 2 and c = 3;
```

这个查询可以定位到 `a = 1` 的数据，而 a = 1 内部的 b 和 c 是有序的，所以能利用索引继续往后匹配。

```
▼sql复制代码where a = 1 and b = 2 and c > 3;
```

这种查询可以完整用上联合索引。因为 a 和 b 都是等值查询，在这个前提下 c 是有序的，有序就能走索引。

> 补充说明：拿 `where a>=1, b=2`，再来理解一下。因为 a = 1 这个条件，再加上 b = 2，mysql 可以直接定位到 a=1 and b=2 这条数据，然后再往后遍历，因此说用上了 b（因为确实利用了 b 的条件快速定位到第一条记录）。但是要注意往后遍历即 扫描 a > 1 的情况是用不到 b 的。



MySQL 8 的 Skip Scan 优化

MySQL 8.0.13 版本引入了 Skip Scan Range Access Method，在特定场景下可以绕过最左匹配原则。

> 以下实验基于 MySQL8.0.26 版本

为了故事顺利的发展，我们先建个表。

```
▼sql复制代码CREATE TABLE `t1` (
  `f1` int NOT NULL AUTO_INCREMENT,
  `f2` int NOT NULL,
  `f3` int NOT NULL,
  PRIMARY KEY (`f1`),
  KEY `idx_f2_f3` (`f2`,`f3`)
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_0900_ai_ci
```

很简单的一张表，一个主键索引，一个联合索引`idx_f2_f3`。

此时插入一波数据

```
▼sql复制代码INSERT INTO t1 (f2,f3) VALUES
  (1,1), (1,2), (1,3), (1,4), (1,5),
  (2,1), (2,2), (2,3), (2,4), (2,5);
INSERT INTO t1(f2,f3) SELECT f2, f3 + 5 FROM t1;
INSERT INTO t1(f2,f3) SELECT f2, f3 + 10 FROM t1;
INSERT INTO t1(f2,f3) SELECT f2, f3 + 20 FROM t1;
INSERT INTO t1(f2,f3) SELECT f2, f3 + 40 FROM t1;
```

这个时候表里一共有160条数据，**f2 的值基数很低，只有1、2这两个值，基数为2。**

数据准备完了，咱们可以开始操作一下了，此时执行：

```
ANALYZE TABLE t1;
EXPLAIN SELECT f2, f3 FROM t1 WHERE f2 = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	t1	NULL	ref	idx_f2_f3	idx_f2_f3	4	const	80	100.00	Using index

面试鸭 mianshiya.com

先执行了一下`ANALYZE TABLE t1;`更新了一下表的统计信息，防止 MySQL 误判。

简单执行了一个 f2 为条件的联合索引查询，且结果只要求返回 f2 和 f3，所以从这个联合索引可以直接得到值，不需要回表。

这个查询的 explain 结果相信大家也都能理解，用上了`idx_f2_f3`，且 type 是ref，说明用上了这个索引查询。

现在我们换一个查询条件，让 f3 作为条件来查询，这个时候就不是最左匹配原则了，讲道理应该用不上索引查询。

```
EXPLAIN SELECT f2, f3 FROM t1 WHERE f3 = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	t1	NULL	range	idx_f2_f3	idx_f2_f3	8	NULL	16	100.00	Using where; Using index for skip scan

面试鸭 mianshiya.com

看看这结果，好像不太对？type 是 range，这是基于索引的范围扫描，所以用上了索引！

但是我的查询条件`f3=1`明明是等值呀？怎么就用上了范围扫描呢？

还有 Extra 里面显示的`Using index for skip scan`又是啥？

Using index for skip scan

让我们直接进行一个官方文档的查询！

- Optimization
  - Optimization Overview
  - Optimizing SQL Statements
    - Optimizing SELECT Statements
      - WHERE Clause Optimization
      - **Range Optimization**
      - Index Merge Optimization
      - Hash Join Optimization
      - Engine Condition Pushdown Optimization
      - Index Condition Pushdown Optimization
      - Nested-Loop Join Algorithms
      - Nested Join Optimization
      - Outer Join Optimization
      - Outer Join Simplification
      - Multi-Range Read Optimization
      - Block Nested-Loop and Batched Key Access Joins
      - Condition Filtering
      - Constant-Folding Optimization
      - IS NULL Optimization
      - ORDER BY Optimization
      - GROUP BY Optimization
      - DISTINCT Optimization
      - LIMIT Query Optimization
      - Function Call Optimization
      - Window Function Optimization
      - Row Constructor Expression Optimization
      - Avoiding Full Table Scans

After execution of a query for which index dives are skipped, the corresponding row in the `INFORMATION_SCHEMA.OPTIMIZER_TRACE` table contains an `index_dives_for_range_access` value of `skipped_due_to_force_index`.

Skip Scan Range Access Method

Consider the following scenario:

```
CREATE TABLE t1 (f1 INT NOT NULL, f2 INT NOT NULL, PRIMARY KEY(f1, f2));
INSERT INTO t1 VALUES
  (1,1), (1,2), (1,3), (1,4), (1,5),
  (2,1), (2,2), (2,3), (2,4), (2,5);
INSERT INTO t1 SELECT f1, f2 + 5 FROM t1;
INSERT INTO t1 SELECT f1, f2 + 10 FROM t1;
INSERT INTO t1 SELECT f1, f2 + 20 FROM t1;
INSERT INTO t1 SELECT f1, f2 + 40 FROM t1;
ANALYZE TABLE t1;

EXPLAIN SELECT f1, f2 FROM t1 WHERE f2 > 40;
```

To execute this query, MySQL can choose an index scan to fetch all rows (the index includes all columns to be selected), then apply the `f2 > 40` condition from the `WHERE` clause to produce the final result set.

A range scan is more efficient than a full index scan, but cannot be used in this case because there is no condition on `f1`, the first index column. However, as of MySQL 8.0.13, the optimizer can perform multiple range scans, one for each value of `f1`, using a method called Skip Scan that is similar to Loose Index Scan (see Section 8.2.1.17, "GROUP BY Optimization"):

1. Skip between distinct values of the first index part, `f1` (the index prefix).
2. Perform a subrange scan on each distinct prefix value for the `f2 > 40` condition on the remaining index part.

For the data set shown earlier, the algorithm operates like this:

1. Get the first distinct value of the first key part (`f1 = 1`).

面试鸭 mianshiya.com

在优化里面的范围优化里确实有`Skip Scan Range Access Method`，它就是产生上述 Extra 的原因。

好了，可以忘了官网这个截图了，基于我展示的例子我来解释下。

根据我们所认知的最左匹配原则，执行

- where f3 =1

是无法在`idx_f2_f3`上利用 type 是 ref 的查询方式来查询的。

因为只有带上索引左边的查询条件才能用上索引，例如：

- where f2=1 and f3 =1

那如果无法利用索引进行查询，最终只有一个选择，全表扫描：要么全表扫描主键索引，要么全表扫描二级索引。

MySQL 就意识到这效率确实低，能不能找个优化点呢？

所以就在 8.0.13 版本引入了`Skip Scan Range Access Method`，它在一定条件下，利用了范围扫描来替代了全表扫描的发生。

具体原理如下：

1. 先统计一下索引最左字段 f2 的基数即唯一值，这里一共有 1 和 2 两个值。
2. 然后将 f2 的值拼入查询条件中构造`where f2=1 and f3=1`和`where f2=2 and f3=1`这样的条件进行查询。

这波是不是小秀？

你 SQL 不带索引最左的条件是吧，我自个儿给你拼上去！

所以一条查询就变成了多次查询，所以 type 就变成了 range 了。

因此最终还是没有逃出最左匹配原则，只是 MySQL 隐式的构造了查询条件，使得看起来好像不需要最左匹配原则而已。

这个优化的中文名叫：**跳过扫描范围访问**(我直译的，哈哈哈)

跳过扫描范围访问的局限性

不要高兴的太早，这个优化其实有很大的局限性！

它要求最左条件的基数很低，因为它需要列举最左条件的值来构造查询，假如最左条件有上万个，那还不如直接进行全表扫描了对吧。

其实也不需要上万个，我来改一下表里的数据来做一次实验。

```
UPDATE t1 SET F2 = 1 + RAND()*500 WHERE f1 > 0;
ANALYZE TABLE t1;
EXPLAIN SELECT f2, f3 FROM t1 WHERE f3 > 1;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	t1	NULL	index	idx_f2_f3	idx_f2_f3	8	NULL	160	33.33	Using where; Using index

面试鸭 mianshiya.com

我直接把 f2 的值随机化了，这样 f2 的基数就变大了，可以看到此时再执行查询语句就用不上了`Using index for skip scan`了。

变成了一个基于二级索引的**全表扫描**。

除了上面这个关键点之外，还有很多局限性，我根据官网的内容做了一下翻译(和一点便于理解的小修改)：

- 需要联合索引
- 查询不能跨表
- 查询不能使用 GROUP BY 或者 DISTINCT
- 查询只能用一个索引，即索引需要覆盖查询的值，不能回表
- 查询条件必须是常量，包括 IN() 运算符。

还有两条我不翻译了，我觉得讲的是废话...有点兴趣的自己去看看吧，文末会放链接。

总而言之，能用上这个优化的场景很有限，就**基数很低**这一个限制我就觉得很大了。

所以这也不是什么万能药，就当一个知识点咱们知晓下，到时候遇到特殊场景能用上的时候装一下，或者在面试官问你最左匹配的时候，提一下，我知道那个 MySQL 8.0.13 版本啊，它做了个优化.....

- range-access-skip-scan

面试官追问

提问：联合索引 (a, b, c)，执行 where b = 1 and a = 2 能用上索引吗？

回答：能用上。MySQL 的查询优化器会自动调整条件顺序，把 a = 2 放到前面，所以写成 `where b = 1 and a = 2` 和 `where a = 2 and b = 1` 效果一样。优化器足够聪明，不用纠结 SQL 里条件的书写顺序。

提问：为什么 >= 不会停止匹配，但 > 会停止匹配？

回答：关键在于能不能精确定位到一条记录。`a >= 1` 可以先精确定位到 `a = 1` 的第一条数据，这时候 b、c 在 a = 1 内部是有序的，可以继续匹配。而 `a > 1` 定位到的是 a = 2 的第一条数据，此时已经跨过了 a = 1 的整个范围，不同 a 值之间 b、c 是无序的，所以后面的列用不上索引。

提问：建联合索引时，列的顺序应该怎么考虑？

回答：一般有几个原则：

1）高频查询条件放左边，保证常用查询能走索引

2）区分度高的列放左边，能更快过滤掉数据

3）范围查询的列放最右边，避免阻断后面列的匹配

4）如果经常有排序需求，把排序字段放到索引里，避免 filesort